

Bash Scripting — Cheat Sheet

From a one-liner to a robust script: structure, variables, conditions, loops, functions, good habits.

Anatomy of a script

```
#!/usr/bin/env bash      # shebang: run this file with bash
set -euo pipefail        # safety belt (see below)

name="world"             # a variable (no spaces around =)
echo "Hello, $name"      # use it with $
```

Run it:

```
chmod +x script.sh      # make it executable (once)
./script.sh              # run it
bash script.sh           # or run without chmod
```

The safety belt: `set -euo pipefail`

Flag	Effect
<code>-e</code>	exit immediately if any command fails
<code>-u</code>	error on use of an undefined variable (catches typos)
<code>-o pipefail</code>	a pipeline fails if any stage fails, not just the last

Put it at the top of every script. It turns silent bugs into loud errors.

Variables

```
count=5                  # assign (NO spaces around =)
echo "$count"            # read (always quote)
greeting="hello world"   # quote values with spaces
readonly PI=3.14          # constant
unset count              # delete a variable
```

Command substitution — capture a command's output:

```
today="$(date +%F)"      # today="2026-09-16"
files="$(ls | wc -l)"    # number of files
```

Arithmetic:

```
n=$(( 3 + 4 ))          # 7
i=$(( i + 1 ))           # increment
```

Rule #1: always quote your variables → `"$var"`. Unquoted variables break on spaces and get glob-expanded.
`rm $file` is a bug; `rm "$file"` is correct.

Parameter expansion (string tricks, no external tools)

Syntax	Result
<code>\${var:-default}</code>	value of <code>var</code> , or <code>default</code> if unset/empty
<code>\${var:=default}</code>	same, and assign the default
<code>\${#var}</code>	length of the string
<code>\${var#pattern}</code>	remove shortest match from the start
<code>\${var##pattern}</code>	remove longest match from the start
<code>\${var%pattern}</code>	remove shortest match from the end
<code>\${var/old/new}</code>	replace first <code>old</code> with <code>new</code>
<code>\${var//old/new}</code>	replace all

Example: `f="/path/to/photo.jpg"` → `${f##*/}` = `photo.jpg`, `${f%.*}` = `/path/to/photo`.

Script arguments

Variable	Means
<code>\$0</code>	the script's name
<code>\$1</code> , <code>\$2</code> , ...	first, second argument
<code>\$@</code>	all arguments (as separate words)
<code>\$#</code>	number of arguments
<code>\$?</code>	exit code of the last command (0 = success)

```
[ -z "$1" ] && { echo "usage: $0 <url>"; exit 1; } # require an argument
url="$1"
```

Conditionals

Use `[[ ... ]]` (bash) — safer than the old `[ ... ]`.

```
if [[ "$age" -ge 18 ]]; then
    echo "adult"
elif [[ "$age" -ge 13 ]]; then
    echo "teen"
else
    echo "child"
fi
```

Number tests: `-eq` `-ne` `-lt` `-le` `-gt` `-ge` (equal, not-equal, less, less-or-equal, greater, greater-or-equal). **String tests:** `=` equal, `!=` not equal, `-z` empty, `-n` non-empty, `=~` regex match. **File tests:** `-e` exists, `-f` is a file, `-d` is a directory, `-r/-w/-x` readable/writable/executable, `-s` non-empty.

```
[[ -f "$path" ]] && echo "file exists"
[[ "$s" =~ ^[0-9]+$ ]] && echo "all digits"
```

Combine with `&&` (and) / `||` (or):

```
mkdir -p out && cd out # cd only if mkdir succeeded
ping -c1 host || echo "unreachable" # message only if ping failed
```

case (cleaner than many elifs)

```
case "$1" in
  start)  echo "starting" ;;
  stop)   echo "stopping" ;;
  *.jpg|*.png) echo "an image" ;;
  *)      echo "unknown: $1" ;;      # default
esac
```

Loops

For, over a list:

```
for name in alice bob carol; do
  echo "hi $name"
done
```

For, over files (glob):

```
for f in *.log; do
  echo "processing $f"
done
```

C-style for:

```
for (( i=0; i<5; i++ )); do echo "$i"; done
```

While (runs while the condition is true):

```
count=0
while [[ $count -lt 3 ]]; do
  echo "$count"; count=$(( count + 1 ))
done
```

Read a file line by line (the correct idiom):

```
while IFS= read -r line; do
  echo "line: $line"
done < input.txt
```

Until (runs until the condition becomes true):

```
until ping -c1 host &>/dev/null; do sleep 2; done
```

Control: `break` leaves the loop, `continue` skips to the next iteration.

Functions

```
greet() {
  local who="$1"      # local: variable stays inside the function
  echo "Hello, $who"
  return 0            # 0 = success
}

greet "world"         # call it
result="$(greet Sam)" # capture its output
```

Arrays

```
fruits=(apple banana cherry)
echo "${fruits[0]}"          # apple
echo "${fruits[@]}"          # all elements
echo "${#fruits[@]}"         # count = 3
fruits+=(date)               # append
for f in "${fruits[@]}"; do echo "$f"; done
```

Reading user input

```
read -r -p "Your name: " name
read -r -s -p "Password: " pass # -s hides typing
echo
```

Here-documents (multi-line text / templates)

```
cat > config.txt <<EOF
user=$USER
date=$(date)
EOF
```

Use `<<'EOF'` (quoted) to keep `$variables` literal (no expansion).

Cleanup with `trap`

```
tmp="$(mktemp)"
trap 'rm -f "$tmp"' EXIT      # always delete tmp on exit, even on error
```

Exit codes

```
command
if [[ $? -eq 0 ]]; then echo "ok"; else echo "failed"; fi
exit 0          # end the script successfully
exit 1          # end with an error
```

Good practices (memorise)

- Start with `#!/usr/bin/env bash` and `set -euo pipefail`.
- **Quote everything:** `"$var"`, `"$@"`, `"$(cmd)"`.
- Prefer `[[ ]]` over `[ ]`, and `$( )` over backticks.
- Use `local` variables inside functions.
- Check arguments before using them; fail early with a clear message.
- Comment the *why*, not the obvious *what*.
- Lint your script with `shellcheck script.sh` — it catches most beginner bugs.
- Make it idempotent: running it twice should be safe.
- Never store secrets in the script; read them from the environment (`"$API_KEY"`).

Minimal robust template

```
#!/usr/bin/env bash
set -euo pipefail

usage() { echo "usage: $0 <input> [output]"; exit 1; }
[[ $# -lt 1 ]] && usage

input="$1"
output="${2:-out.txt}"          # default if not given

[[ -f "$input" ]] || { echo "no such file: $input" >&2; exit 1; }

while IFS= read -r line; do
    echo "processed: $line"
done < "$input" > "$output"

echo "done -> $output"
```